



POWERPAY

Solana-native PWRC Checkout & Settlement

Technical Whitepaper

Canonical release

Version 1.0.0 • Devnet + Mainnet Beta • PWRC Token-2022 • Solana Pay • Browser wallet settlement

Canonical PWRC mint

PWRCRXXZxbg6FdQZfK3PMD7KP8xfxs9acvifjiG46wc



Powered by Solana

1

2

3

SelectPayComplete



Buy PWRC

PowerChain Token on Solana

You pay

 SOL

0.50

≈ \$73.50 USD

1 SOL ≈ \$147.00 USD

You receive

 PWRC

36,750,000 PWRC

1 PWRC ≈ \$0.000002 USD

Exchange rate

1 SOL = 73,500,000 PWRC

Rate updates in real time

View details

Choose payment method

☒  Solana Pay

Fast • Secure • Native

Recommended

☐  Solana Wallet

Phantom, Solflare, Backpack & more



☐  Card Payment

Visa, MasterCard, Stripe



☒ Your payment is secure and encrypted

Secure Checkout

Powered by Solana

Scan To Pay

Pay securely with Solana Pay



Amount

0.50 SOL

To

7xKQ...9d3F

Network

 Solana

1 Open your Solana wallet

2 Scan the QR code

3 Approve the payment

4 Receive PWRC instantly

Payment expires in

04 : 57

min sec

Order Summary

You pay

0.50 SOL

You receive

36,750,000 PWRC

Network fee (est.)

0.00001 SOL

Total

0.50 SOL

≈ \$73.50 USD

No hidden fees. You pay exactly 0.50 SOL to receive 36,750,000 PWRC.

Powering a Renewable Future

Invest in clean energy. Earn value. Build a better planet.

Explore all products



Solar Energy

Clean • Efficient

Learn more



Wind Energy

Sustainable • Reliable

Learn more



Energy Storage

Store • Optimize

Learn more



Hydro Power

Renewable • Powerful

Learn more



EV Charging

Green • Connected

Learn more

 Built on Solana

Fast • Scalable • Low Fees

 Secure & Trusted

Bank-grade security

 Instant Settlement

Powered by Solana

 Better Planet

100% Renewable Focus

© 2024 PowerChain. All rights reserved.

Terms of Service | Privacy Policy | Support

Illustrative checkout interface. Live rates, balances, and network fees are resolved at runtime.

Contents

01	Executive summary
02	Product thesis and design principles
03	System architecture
04	Checkout and settlement flow
05	Fee architecture
06	PWRC Token-2022 integration
07	Solana Pay and wallet execution
08	Network and RPC model
09	Price and market-data layer
10	Program architecture and receipts
11	Security and privacy controls
12	Operational readiness and observability
13	Developer and deployment model
14	Risks, limitations and legal considerations
15	Roadmap
16	Conclusion

Document status

This whitepaper describes the canonical PowerPay 1.0.0 architecture. It is a technical system description, not an offer of securities, investment advice, or a guarantee of token value.

Executive summary

PowerPay is a Solana-native checkout and settlement application for acquiring PWRC with SOL. It combines a browser-based purchase experience, Wallet Standard connectivity, Solana Pay Scan To Pay, cluster-aware send/receive flows, reference market data, and an Anchor program that enforces the executable sale terms on-chain.

The core design principle is separation of authority. User-interface state, RPC providers, market-data feeds, and convenience services can improve usability, but none of them may redefine the executable PWRC rate, treasury destination, mint identity, fee policy, purchase limits, or settlement receipt. Those controls remain anchored in reviewed configuration and the deployed PowerPay program.

- Canonical PWRC Token-2022 mint: PWRCRXXZxbg6FdQZfK3PMD7KP8xfxs9acvifjiG46wc.
- Two public execution environments: Devnet for integration testing and Mainnet Beta for production settlement.
- Fee transparency: 2% PowerPay service fee in SOL, 2% PWRC Token-2022 transfer fee, and a separate Solana network fee.
- Quote-bound settlement: buyers sign expected rate, fee policy, and minimum net PWRC constraints.
- Replay-resistant receipts: each purchase reference creates a program-owned PurchaseReceipt PDA.
- Pyth and Birdeye provide SOL/USD reference data only; they do not determine the executable PWRC-per-SOL sale rate.
- Fail-closed production posture: Mainnet checkout does not fall back to an executable preview when live on-chain quote state is unavailable.

Value proposition

A checkout should be simple for the buyer and exact for the protocol. PowerPay is designed so the front end can evolve without weakening the settlement boundary.

Product thesis and design principles

One checkout, one explicit settlement path

PowerPay narrows the payment problem: the buyer selects a supported Solana environment, reviews the amount of SOL being spent, sees the service and token-fee effects, connects or opens a compatible wallet, signs the transaction, and receives PWRC through the Token-2022 program. There is no hidden server-side balance mutation that substitutes for chain settlement.

Design principles

Principle	Implementation
Settlement authority	The Anchor program, SaleConfig PDA, canonical mint, and Solana runtime govern execution.
User sovereignty	Wallet connection does not authorize spending; every purchase or transfer requires an explicit wallet signature.
Network clarity	Devnet is marked TEST; Mainnet Beta is marked LIVE and requires explicit confirmation.
Fee legibility	Service fee, Token-2022 fee, and Solana network fee are presented as separate concepts.
Fail-closed production	Mainnet requires live reviewed configuration and on-chain quote availability.
Minimal trust in display data	Pyth/Birdeye drive reference USD presentation, not executable token settlement.
Accessible interaction	Dialogs trap focus, close on Escape, restore focus, lock background scroll, and preserve route-aware aria-current.

User surfaces

- Buy PWRC checkout
- Solana Pay Scan To Pay
- Wallet connect and account menu
- Send SOL / PWRC
- Receive address and QR
- Network selector
- System-health and RPC status
- Terms of Sale, Cookies, and Disclaimer

System architecture

PowerPay is divided into presentation, wallet, API, RPC, reference-data, and on-chain settlement layers. Each layer has a bounded responsibility and explicit failure behavior.

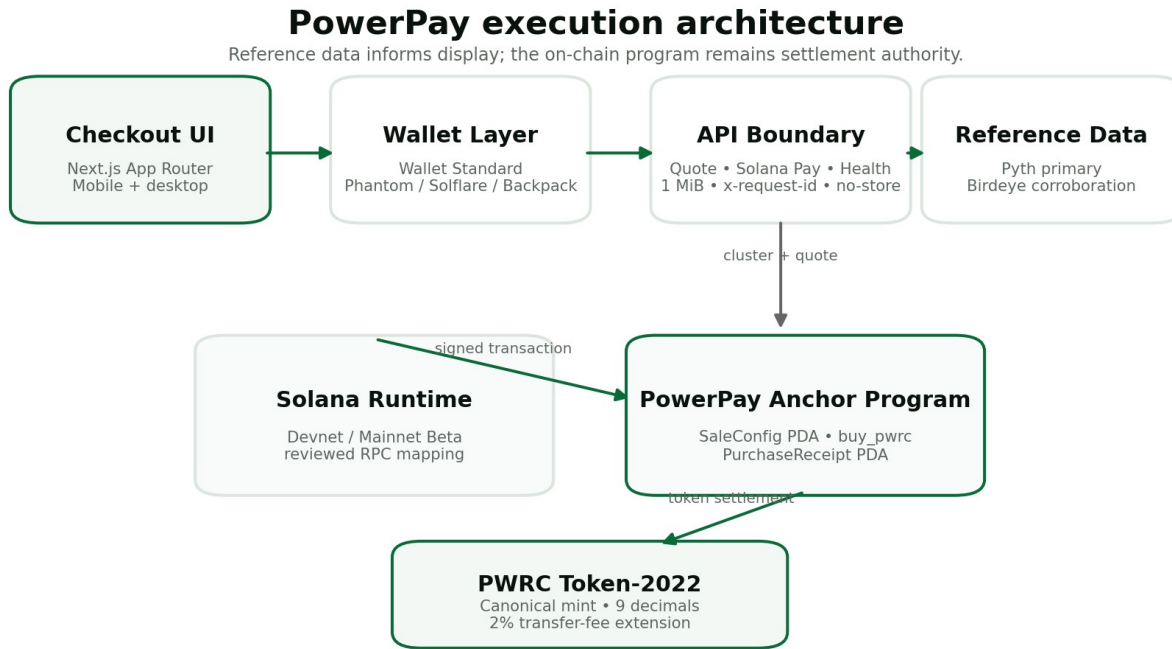


Figure 1 — PowerPay execution architecture.

Trust boundaries

Boundary	Trusted for	Not trusted for
Browser UI	Rendering quote state, collecting user intent, initiating wallet actions	Defining executable token rate or treasury
Wallet	Signing the transaction chosen by the user	Changing program rules
Server/API	Cluster mapping, transaction construction, status verification	Bypassing program constraints
RPC	Transport to the selected Solana cluster	Changing settlement authority
Pyth/Birdeye	Reference SOL/USD display and market-data health	Executable PWR-per-SOL rate
Anchor program	Sale rules, fee validation, asset movement, receipt creation	Off-chain USD valuation

Checkout and settlement flow

A valid purchase is an atomic Solana transaction that carries buyer-reviewed parameters into the program. The program rejects stale or inconsistent execution terms before value is moved.

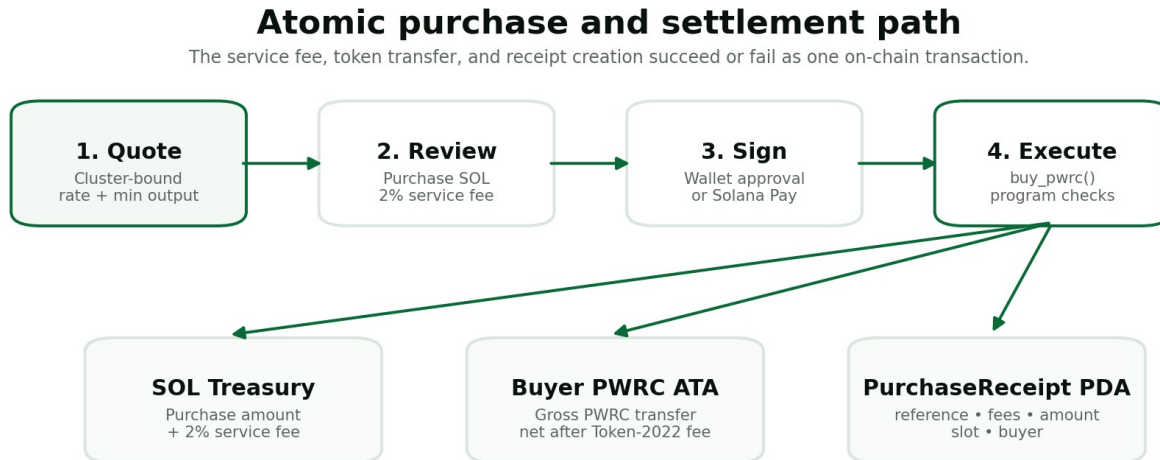


Figure 2 — Quote, approval, execution, and receipt creation.

Quote-bound instruction

The purchase instruction binds the SOL amount, expected gross PWRC-per-SOL rate, minimum acceptable net PWRC, and expected Token-2022 transfer-fee basis points. A production implementation may additionally bind service-fee expectations where required by the active program layout.

Program-side checks

- Sale enabled state and purchase amount limits.
- Canonical PWRC mint and 9-decimal configuration.
- Token-2022 transfer-fee extension exists and active fee equals 200 bps.
- Buyer-reviewed rate matches the executable SaleConfig rate.
- Minimum net PWRC constraint remains satisfied.
- Configured sale vault contains sufficient inventory.
- Purchase reference has not already produced a receipt.

Atomicity

The SOL transfer, service-fee transfer, PWRC transfer, and PurchaseReceipt creation are designed to succeed or fail within one program-controlled transaction. Partial application state is not treated as settlement.

Fee architecture

PowerPay separates three fee concepts so buyers and operators can reason about cost without conflating protocol fees with network costs.

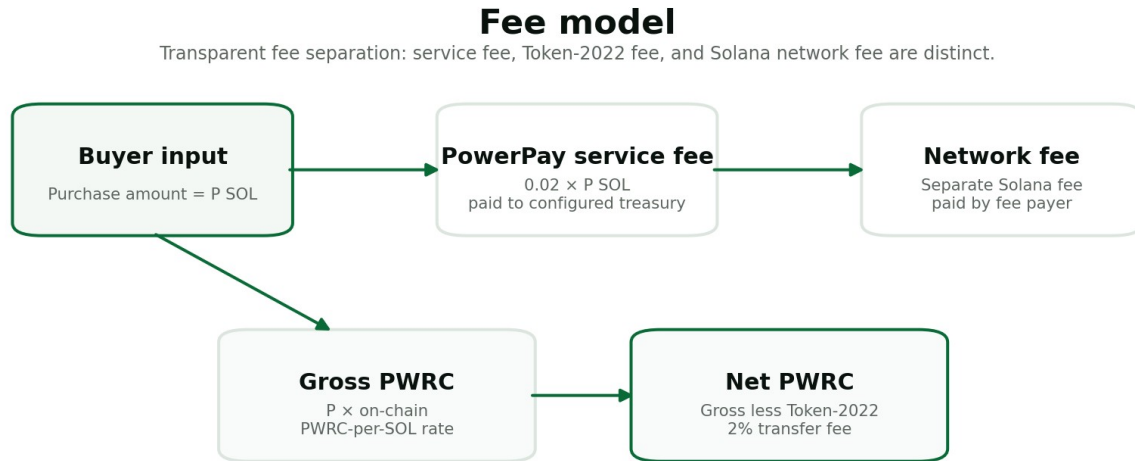


Figure 3 — PowerPay fee model.

Fee	Canonical policy	Settlement treatment
PowerPay service fee	2% / 200 bps of purchase SOL	Transferred atomically to the configured treasury with the purchase.
PWRC transfer fee	2% / 200 bps under Token-2022	Applied by Token-2022 to the PWRC transfer, subject to mint configuration and max-fee semantics.
Solana network fee	Variable	Paid separately by the transaction fee payer to the Solana network.

Calculation model

Let P be the user-selected purchase amount in SOL. The PowerPay service fee is $0.02 \times P$ SOL. The gross PWRC output is P multiplied by the executable on-chain PWRC-per-SOL rate. The net PWRC received is the Token-2022 transfer result after the configured transfer-fee extension is applied. The Solana network fee is independent of both calculations.

No price-feed substitution

Reference SOL/USD prices can change the fiat display, but they do not alter P , the on-chain PWRC rate, or the program-defined settlement rules.

PWRC Token-2022 integration

PWRC is integrated as a canonical Solana Token-2022 asset. PowerPay pins the mint address in application and program policy rather than accepting a caller-supplied token mint.

Canonical mint

PWRCRXXZxbg6FdQZfK3PMD7KP8xfxs9acvifjiG46wc

Invariant	Requirement
Program owner	Token-2022 program
Decimals	9
Transfer fee extension	Required
Active transfer fee	200 bps / 2%
Mint identity	Canonical address only
Buyer account	Canonical associated token account for PWRC
Sale inventory	Program-controlled sale vault boundary

Why Token-2022 matters

Token-2022 allows the mint to express protocol-level extensions such as transfer fees without requiring the checkout to invent an off-chain substitute. PowerPay therefore validates the active extension state at settlement time and surfaces the economic effect in the quote UX.

Mint and upgrade risk

Any material change to mint authority, extension configuration, fee basis points, or max-fee policy can alter the economic behavior of transfers. Production operations should treat such changes as governance-level events requiring independent verification, documented approval, and regression testing before checkout enablement.

Solana Pay and wallet execution

Wallet Standard connection

PowerPay discovers compatible Solana wallets through Wallet Standard and presents a PowerPay-owned connect modal. The modal distinguishes connection from transaction authorization: connecting exposes an address and signing capability but does not authorize spending.

- Phantom, Solflare, Backpack and other Wallet Standard-compatible wallets can be surfaced when installed.
- Connected-wallet balances are cluster-aware and used for UX sufficiency checks only.
- Changing cluster disconnects the wallet before the connection provider is remounted.
- Dialogs implement focus trapping, Escape-to-close, focus restoration and background scroll locking.

Solana Pay Scan To Pay

Scan To Pay creates an expiring transaction-request flow bound to the selected network and purchase reference. The QR is a transport for wallet intent; settlement status is confirmed from program-owned PurchaseReceipt state rather than by merely finding a reference key in an arbitrary transaction.

Step	Action
1	Generate cluster-bound Solana Pay request and unique purchase reference.
2	Buyer opens a Solana wallet or scans the QR code.
3	Wallet retrieves and displays the serialized transaction for review.
4	Buyer signs and submits.
5	Program executes sale invariants and creates the PurchaseReceipt PDA.
6	Status API verifies the program-owned receipt and returns confirmed settlement state.

Network and RPC model

PowerPay supports exactly two public Solana runtime modes: Devnet and Mainnet Beta. Arbitrary RPC URLs and program IDs are not accepted from public requests. The client sends a cluster identifier; server code resolves that identifier to reviewed environment configuration.

Mode	Purpose	Safety posture
Devnet	Integration, wallet, QR, program and replay testing	TEST badge; preview behavior may be permitted when explicitly configured.
Mainnet Beta	Production settlement with real SOL/PWRC	LIVE badge; explicit confirmation; no executable preview if live quote state is unavailable.

RPC UX and hardening

- 8-second timeout for RPC checks.
- Defensive JSON parsing and malformed-response handling.
- 1 MiB response ceiling for protected RPC reads.
- Latency and explicit checking/ok/error states exposed to the UI.
- aria-busy used while checks are in progress.
- Clear user messaging that changing the read RPC or network cannot override treasury, mint, sale program, or settlement authority.

Mainnet enablement

Mainnet Beta should remain disabled until the deployed program, treasury, canonical mint, RPC endpoints, sale inventory, Token-2022 fee state, and PurchaseReceipt flow have been independently verified.

Price and market-data layer

PowerPay uses Pyth as the primary SOL/USD reference source and Birdeye as corroboration/fallback data. This layer exists to improve price display, freshness signaling, and operator awareness; it does not govern the executable PWRC rate.

Concern	Policy
Primary source	Pyth SOL/USD reference data.
Corroboration	Birdeye reference data where configured and available.
Freshness	Stale data is labeled and may be rejected for display-sensitive flows.
Divergence	Cross-source deviation can be measured in basis points and surfaced as a warning.
Secrets	Provider API keys remain server-only and must never use NEXT_PUBLIC_* variables.
Settlement boundary	SaleConfig remains authoritative for executable PWRC-per-SOL output.

Failure behavior

If market data is unavailable, PowerPay can degrade reference USD display without inventing a substitute settlement rate. If the Mainnet executable quote itself is unavailable, the purchase path should fail closed. This distinction prevents an oracle outage from silently becoming a token-pricing decision.

Program architecture and receipts

SaleConfig PDA

SaleConfig is the on-chain control plane for the active sale. It defines the authority, SOL treasury, PWRC mint, gross PWRC-per-SOL rate, minimum and maximum purchase amounts, enabled state, and sale-vault boundary.

PurchaseReceipt PDA

Each purchase reference derives a program-owned receipt PDA using a deterministic seed. Creating the receipt with init makes replay attempts fail because the same reference cannot initialize the same PDA twice.

Receipt evidence	Purpose
Buyer	Binds settlement to the purchasing wallet.
Reference	Provides order identity and replay protection.
Purchase lamports	Records the base SOL purchase amount.
Service fee	Records the fee amount/policy applied at execution.
Gross / net PWRC	Provides token-side settlement evidence.
Transfer-fee bps	Proves the expected Token-2022 fee policy.
Settlement slot	Anchors receipt to Solana execution time/order.

Authority operations

Initialization, funding, rate changes, limit changes, treasury changes, pause/enable actions, and program upgrades are privileged operations. They should be performed with cluster-explicit administration commands, reviewed addresses, and separate Mainnet change approval.

Security and privacy controls

Application boundary

Control	Policy
Request body ceiling	1 MiB maximum for normal public JSON handlers.
Request correlation	Sanitized or server-generated x-request-id.
Cache policy	Cache-Control: no-store, max-age=0 and Pragma: no-cache.
Error model	Typed application errors with stable HTTP status behavior.
Robots/privacy	X-Robots-Tag: noindex, nofollow, noarchive on protected application surfaces.
Security headers	nosniff, restrictive referrer policy, permissions policy.
OpenAPI/Swagger	Authorization persistence disabled; Try It Out disabled by default.
WebSocket guard	64 KiB messages, auth-failure cap, 32 subscriptions, 120 messages/minute, policy/rate close codes.

Dependency and supply-chain posture

The release process treats dependency build scripts and transitive advisories as security boundaries. pnpm build-script approvals are explicit and new executable install scripts fail closed until reviewed. Vulnerable transitive chains should be removed or converged through reviewed dependencies rather than masked by unsupported overrides.

- Regenerate and commit pnpm-lock.yaml after dependency changes.
- Run dependency-security checks and pnpm audit before release.
- Reject known vulnerable dependency ranges in CI where practical.
- Prefer browser-specific dependency graphs that avoid unused React Native/Metro toolchains.
- Keep Mainnet enablement independent from successful front-end compilation.

Accessibility as a security-adjacent control

Accessible dialogs and predictable keyboard behavior reduce accidental actions in high-value flows. PowerPay therefore preserves focus, supports Escape dismissal, locks background scrolling during modal interaction, and marks the active route with aria-current.

Operational readiness and observability

A production checkout needs runtime evidence that the selected environment is healthy without exposing sensitive infrastructure details. PowerPay provides a system-health surface that can validate RPC reachability, deployed program state, SaleConfig initialization, canonical mint ownership, active Token-2022 fee policy, sale enabled state, and inventory availability.

Signal	Meaning
Operational	RPC reachable and required program/sale prerequisites present.
Sale paused	Infrastructure is available but the sale is intentionally disabled.
Needs attention	One or more runtime checks failed or required state is unavailable.

Operational separation

- Health state is informational; it does not override transaction-builder or program validation.
- Private RPC URLs and provider credentials are not returned to the browser.
- Wallet balances are UX hints; the Solana runtime remains authoritative at signing time.
- Mainnet initialization, funding and enablement should be independently approved operations.
- Explorer links and receipt verification must be cluster-correct.

Developer and deployment model

PowerPay is organized as a pnpm workspace with a Next.js web application and Anchor programs. The canonical release identifier remains 1.0.0 while implementation improvements can advance within that release contract until a deliberate versioning decision is made.

Layer	Representative technology
Web	Next.js 16, React 19, TypeScript 5.9, Tailwind CSS
Wallet/Solana client	Wallet Standard adapters, @solana/web3.js, @solana/pay, @solana/kit
On-chain	Anchor 1.1.x / Rust / Token-2022
Market data	Pyth + Birdeye reference pricing
Package management	pnpm 11 with explicit build-script policy
Testing	Static architecture/security/a11y checks, TypeScript, Anchor build/test

Release gates

- pnpm install --frozen-lockfile
- pnpm run doctor
- dependency-security, accessibility, architecture and security regression checks
- pnpm typecheck
- pnpm build
- anchor build
- anchor test
- Devnet end-to-end purchase and Solana Pay verification
- Mainnet controlled-value smoke test only after audit/approval

Deployment rule

Do not enable Mainnet merely because the UI builds. Production activation requires verified program identity, canonical mint state, treasury, sale configuration, inventory, RPC infrastructure, and end-to-end receipt verification.

Risks, limitations and legal considerations

PowerPay interacts with public blockchain infrastructure and digital assets. Users and operators should understand that transaction finality, asset values, wallets, RPC providers, smart contracts, market-data services, and regulatory requirements can all introduce risk.

Risk	Consideration
Blockchain finality	Submitted Mainnet transactions may be irreversible after confirmation.
Wallet security	PowerPay never needs a seed phrase; compromised wallets can still authorize malicious transactions outside PowerPay.
Program risk	Smart-contract defects, upgrade-authority compromise, or misconfiguration can cause loss.
Token economics	PWRC price, liquidity and utility can change; no value appreciation is guaranteed.
Oracle/reference data	Reference feeds can be stale, unavailable or divergent and must not be treated as guaranteed valuation.
RPC/provider outage	Connectivity failures can delay quote reads, submission or status verification.
Jurisdiction	Digital-asset sales and token functionality may be restricted or regulated depending on location.
Third-party services	Wallets, Solana Pay clients, explorers, Pyth, Birdeye and infrastructure providers have independent terms and risks.

Disclaimer

This document is technical documentation only. It is not financial, investment, tax, accounting, or legal advice; it is not a prospectus or an offer to sell securities. PWRC utility, transferability, fees, availability, and access may be limited by technical or legal constraints. Users should review the applicable Terms of Sale and independent professional advice before transacting.

Roadmap

Future work should deepen settlement assurance and operational maturity without weakening the simplicity of the checkout.

Phase	Priority
Hardening	Independent program review, stricter CI dependency gates, rate/fee invariant tests, incident runbooks.
Payments	Merchant-ready checkout intents, richer receipts, optional invoice references, stronger reconciliation.
Data	Provider health scoring, market-data provenance, historical rate audit records.
Operations	Mainnet rollout controls, treasury observability, alerts and deployment attestations.
UX	More compact mobile purchase review, progressive fee disclosure, richer post-settlement receipts.
Ecosystem	Renewable-energy storefront integrations and PowerChain application embedding.

Roadmap principle

New payment rails or data providers should enter through explicit adapters. They must not bypass the PowerPay program, canonical mint checks, quote constraints, or receipt model.

Conclusion

PowerPay 1.0.0 is designed as a focused bridge between familiar checkout UX and explicit Solana settlement. Its architecture makes the critical rules legible: the buyer controls the wallet signature; the selected Solana cluster controls execution; the sale program controls the rate and treasury; Token-2022 controls the PWRC transfer-fee behavior; and PurchaseReceipt PDAs provide deterministic settlement evidence.

The product can therefore improve presentation, market-data quality, wallet discovery, and operational tooling without granting those layers the authority to rewrite settlement. That separation is the foundation for scaling PowerPay from Devnet testing into controlled Mainnet production.

Canonical release

PowerPay 1.0.0 • Solana-native PWRC checkout • Devnet + Mainnet Beta • Solana Pay • Token-2022 fee-aware settlement

Selected technical references

- Solana documentation — transactions, clusters, associated token accounts and RPC.
- Solana Pay specification — transaction-request QR flows.
- Solana Token-2022 documentation — transfer-fee extension behavior.
- Anchor documentation — account constraints, program deployment and IDL/client workflows.
- Pyth Network documentation — price-feed retrieval and freshness considerations.
- Birdeye documentation — Solana market-data API usage.
- PowerPay repository docs — ARCHITECTURE.md, FEES.md, NETWORKS.md, PROGRAM_SECURITY.md, SECURITY.md and RELEASE_CHECKLIST.md.